Industrial Internship Report on

”Banking Information System”

Prepared by

Govarthan

Executive Summary
This report provides details of the Industrial Internship provided by upskill Campus and The IoT Academy in collaboration with Industrial Partner UniConverge Technologies Pvt Ltd (UCT). This internship was focused on a project/problem statement provided by UCT. We had to finish the project including the report in 6 weeks’ time. My project was to design and develop a Banking Information System in Core Java. It is a console-based application that creates savings and current accounts and performs deposit, withdrawal, balance check and account display operations, with validation and exception handling for wrong inputs. This internship gave me a very good opportunity to get exposure to Industrial problems and design/implement solution for that. It was an overall great experience to have this internship.

TABLE OF CONTENTS

1 Preface....3
2 Introduction....5
2.1 About UniConverge Technologies Pvt Ltd....5
2.2 About upskill Campus (USC)....9
2.3 The IoT Academy....10
2.4 Objectives of this Internship program....10
2.5 Reference....10
2.6 Glossary....11
3 Problem Statement....12
4 Existing and Proposed solution....13
4.1 Code submission (Github link)....13
4.2 Report submission (Github link)....13
5 Proposed Design/ Model....13
5.1 High Level Diagram (if applicable)....14
5.2 Low Level Diagram (if applicable)....15
5.3 Interfaces (if applicable)....15
6 Performance Test....16
6.1 Test Plan/ Test Cases....17
6.2 Test Procedure....17
6.3 Performance Outcome....18
7 My learnings....19
8 Future work scope....19

1 Preface

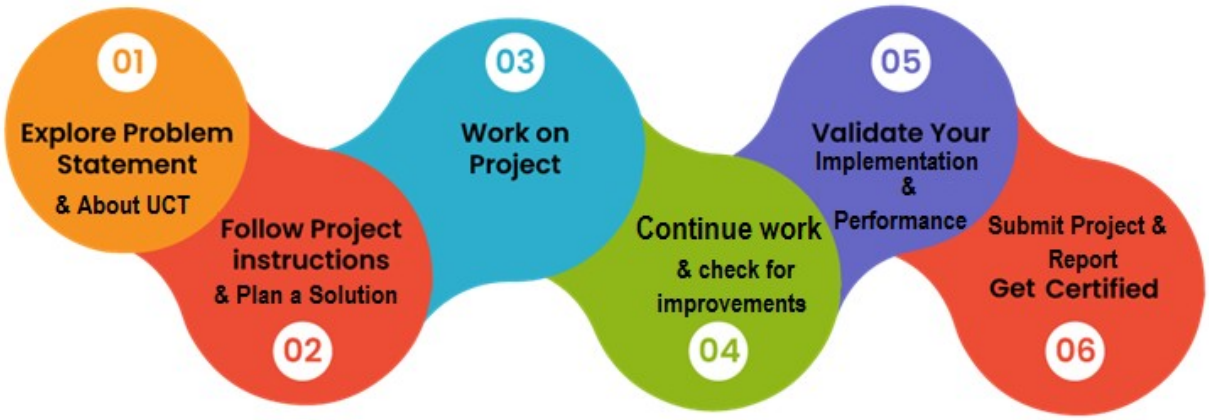
Summary of the whole 6 weeks' work: During this internship I studied the core concepts of Java step by step, including platform independence, loops and decisions, methods, classes and objects, interfaces, abstract classes, method overloading and exception handling. I then applied these concepts to build the Banking Information System.

Need of a relevant internship: An internship connects what we learn in class with how problems are solved in industry, builds practical skills and confidence, and improves job prospects.

About my project: The project is a Banking Information System written in Core Java. It manages bank accounts and supports creating accounts, depositing, withdrawing, checking balance and displaying account details.

Opportunity given by USC/UCT: upskill Campus and The IoT Academy, in association with UniConverge Technologies Pvt Ltd (UCT), gave me the chance to work on an industrial project with a structured learning path, weekly quizzes and weekly progress reports.

How Program was planned



My learnings and overall experience: I learned to write object-oriented Java programs, divide a problem into classes and methods, and test a program with different inputs. Overall, it was a very good learning experience.

Thanks: I thank upskill Campus, The IoT Academy and UniConverge Technologies Pvt Ltd (UCT), and my mentors, friends and family who supported me directly or indirectly.

Message to my juniors and peers: Start early, practise by writing code yourself, and learn from the mistakes in every quiz.

2 Introduction

2.1 About UniConverge Technologies Pvt Ltd

A company established in 2013 and working in Digital Transformation domain and providing Industrial solutions with prime focus on sustainability and RoI.

For developing its products and solutions it is leveraging various **Cutting Edge Technologies e.g. Internet of Things (IoT), Cyber Security, Cloud computing (AWS, Azure), Machine Learning, Communication Technologies (4G/5G/LoRaWAN), Java Full Stack, Python, Front end** etc.



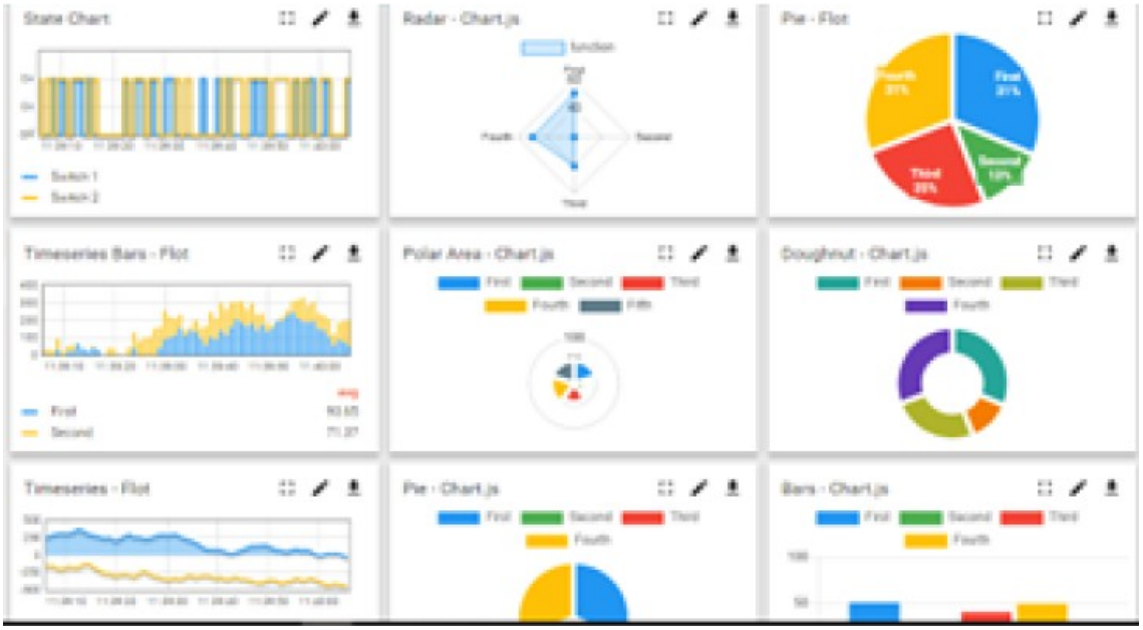
i. UCT IoT Platform (Insight)

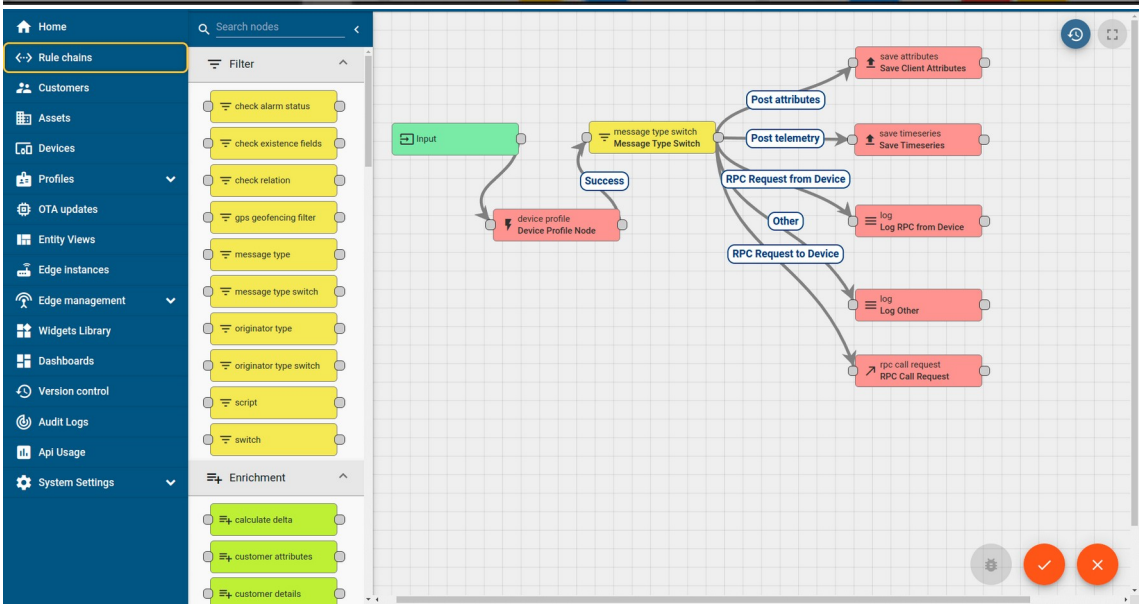
UCT Insight is an IOT platform designed for quick deployment of IOT applications on the same time providing valuable “insight” for your process/business. It has been built in Java for backend and ReactJS for Front end. It has support for MySQL and various NoSql Databases.

- It enables device connectivity via industry standard IoT protocols - MQTT, CoAP, HTTP, Modbus TCP, OPC UA
- It supports both cloud and on-premises deployments.

It has features to

- Build Your own dashboard
- Analytics and Reporting
- Alert and Notification
- Integration with third party application(Power BI, SAP, ERP)
- Rule Engine





ii. Smart Factory Platform (FACTORY WATCH)

Factory watch is a platform for smart factory needs.

It provides Users/ Factory

- with a scalable solution for their Production and asset monitoring
- OEE and predictive maintenance solution scaling up to digital twin for your assets.
- to unleased the true potential of the data that their machines are generating and helps to identify the KPIs and also improve them.
- A modular architecture that allows users to choose the service that they what to start and then can scale to more complex solutions as per their demands.

Its unique SaaS model helps users to save time, cost and money.



Machine	Operator	Work Order ID	Job ID	Job Performance	Job Progress		Output		Rejection	Time (mins)				Job Status	End Customer
					Start Time	End Time	Planned	Actual		Setup	Pred	Downtime	Idle		
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i

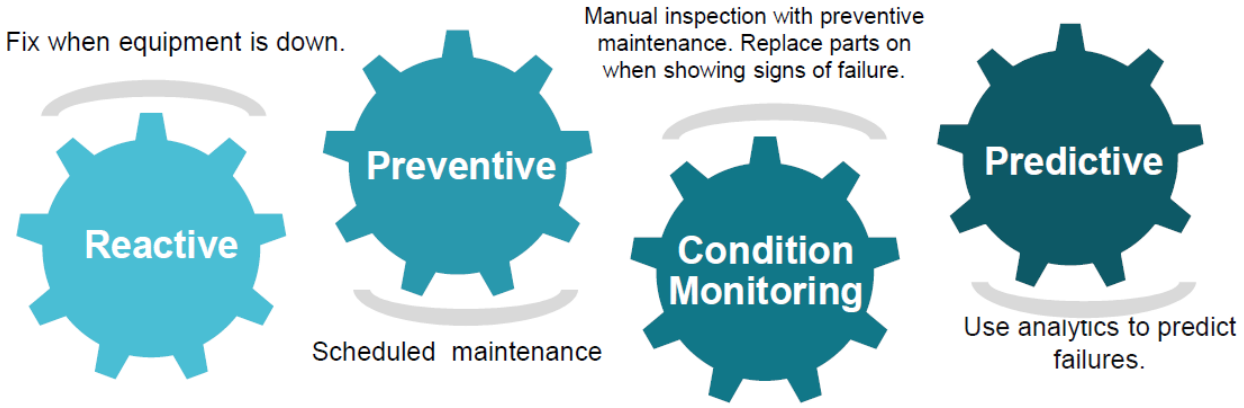


iii. LoRaWAN based Solution

UCT is one of the early adopters of LoRAWAN teschnology and providing solution in Agritech, Smart cities, Industrial Monitoring, Smart Street Light, Smart Water/ Gas/ Electricity metering solutions etc.

iv. Predictive Maintenance

UCT is providing Industrial Machine health monitoring and Predictive maintenance solution leveraging Embedded system, Industrial IoT and Machine Learning Technologies by finding Remaining useful life time of various Machines used in production process.



2.2 About upskill Campus (USC)

upskill Campus along with The IoT Academy and in association with Uniconverge technologies has facilitated the smooth execution of the complete internship process.

USC is a career development platform that delivers **personalized executive coaching** in a more affordable, scalable and measurable way.





2.3 The IoT Academy

The IoT academy is EdTech Division of UCT that is running long executive certification programs in collaboration with EICT Academy, IITK, IITR and IITG in multiple domains.

2.4 Objectives of this Internship program

The objective for this internship program was to

- ☛ get practical experience of working in the industry.
- ☛ to solve real world problems.
- ☛ to have improved job prospects.
- ☛ to have Improved understanding of our field and its applications.
- ☛ to have Personal growth like better communication and problem solving.

2.5 Reference

[1] D. J. Eck, Introduction to Programming Using Java (course textbook).

[2] Oracle, Java Platform, Standard Edition documentation, https://docs.oracle.com/en/java/

[3] upskill Campus, Core Java Internship course material and weekly quizzes.

2.6 Glossary

Terms	Acronym
JVM	Java Virtual Machine
JDK	Java Development Kit
OOP	Object Oriented Programming
API	Application Programming Interface
UCT	UniConverge Technologies Pvt Ltd

3 Problem Statement

In the assigned problem statement, we had to design and develop a Banking Information System using Core Java.

The system should store the details of bank customers and let the user perform the basic banking operations on their accounts. The main requirements were:

- Create savings and current accounts with an account number, holder name and opening balance.
- Deposit money into and withdraw money from an account.
- Maintain a minimum balance for savings accounts.
- Check the balance and display the details of one account or of all accounts.
- Handle wrong inputs, such as invalid amounts or unknown account numbers, without crashing.

4 Existing and Proposed solution

Existing solutions: Small institutions often keep account records in paper registers or spreadsheets, while large banks use commercial core banking software. Manual records are slow and error-prone, spreadsheets do not enforce rules such as a minimum balance, and commercial software is complex, costly and needs training, so it is not suitable for learning or small needs.

Proposed solution: A simple console-based Banking Information System in Core Java. A menu lets the user choose an operation, and the program applies the banking rules automatically.

Value addition: The program is built with an object-oriented design (an interface, an abstract class and two account subclasses). It checks every input and shows clear error messages, and it is easy to extend with new account types or features.

4.1 Code submission (Github link)

https://github.com/govarthan-AI/upskillcampus/blob/main/BankingInformationSystem.java

4.2 Report submission (Github link)

https://github.com/govarthan-AI/upskillcampus/blob/main/BankingInformationSystem_Govarthan_USC_UCT.pdf

5 Proposed Design/ Model

The design flow of the solution has a start, intermediate stages and a final outcome. The user selects an option from the console menu. The program validates the input, runs the matching account operation on an account object, updates the list of accounts held in memory, and prints the result or an error message.

5.1 High Level Diagram (if applicable)

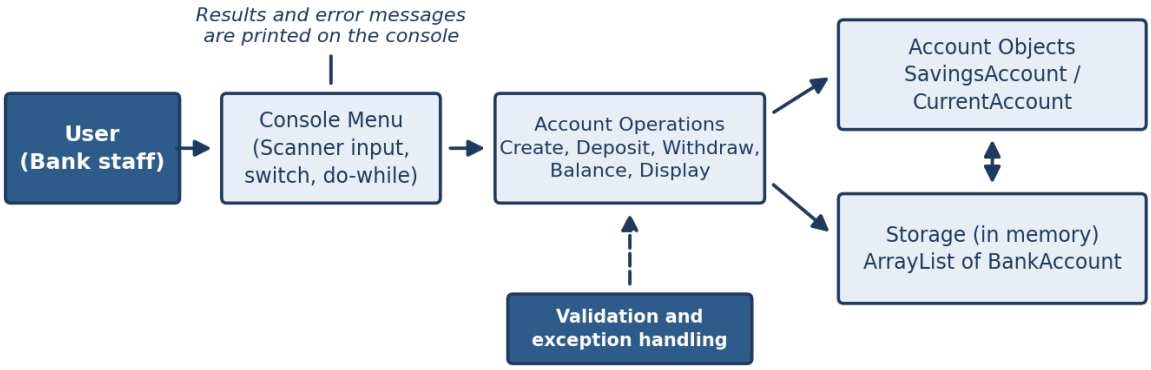


Figure 1: HIGH LEVEL DIAGRAM OF THE SYSTEM

5.2 Low Level Diagram (if applicable)

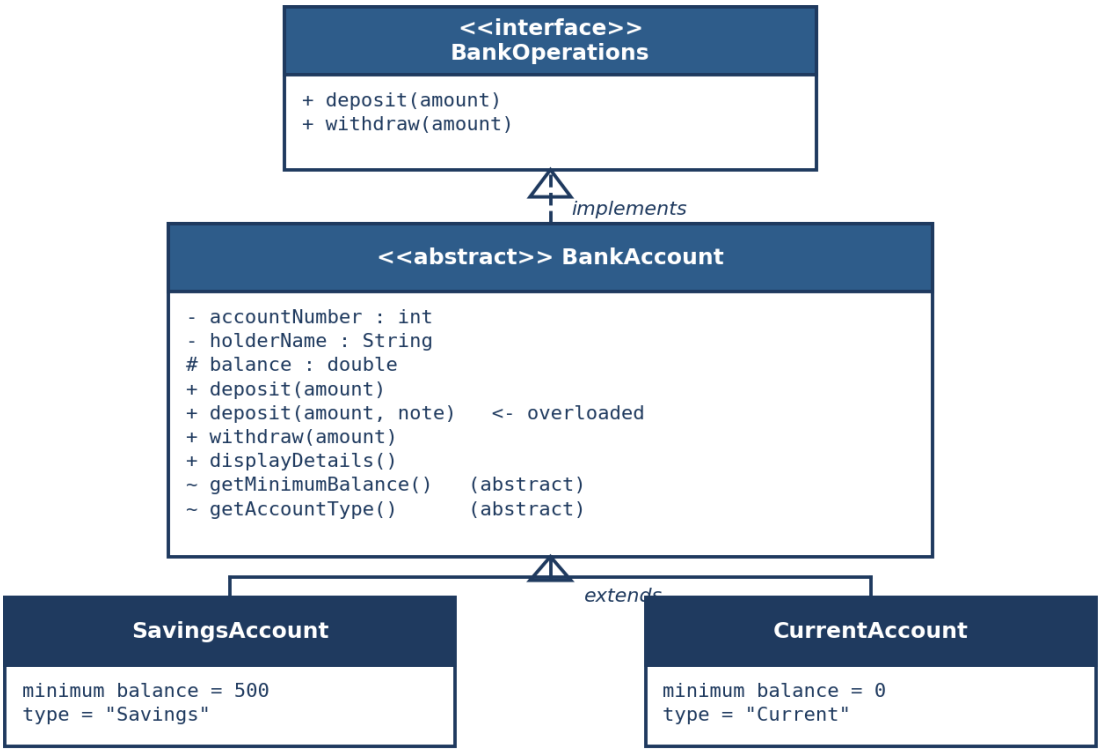


Figure 2: LOW LEVEL (CLASS) DIAGRAM OF THE SYSTEM

5.3 Interfaces (if applicable)

User interface: The program has a console menu with seven options: 1) Create Account, 2) Deposit Money, 3) Withdraw Money, 4) Check Balance, 5) Display Account Details, 6) Display All Accounts, 7) Exit. The user types the option number and the required details, and the program prints the result.

Software interface: The interface BankOperations declares deposit() and withdraw(). The abstract class BankAccount implements it, and SavingsAccount and CurrentAccount extend BankAccount. The flow chart in Figure 3 shows how the menu loop works.

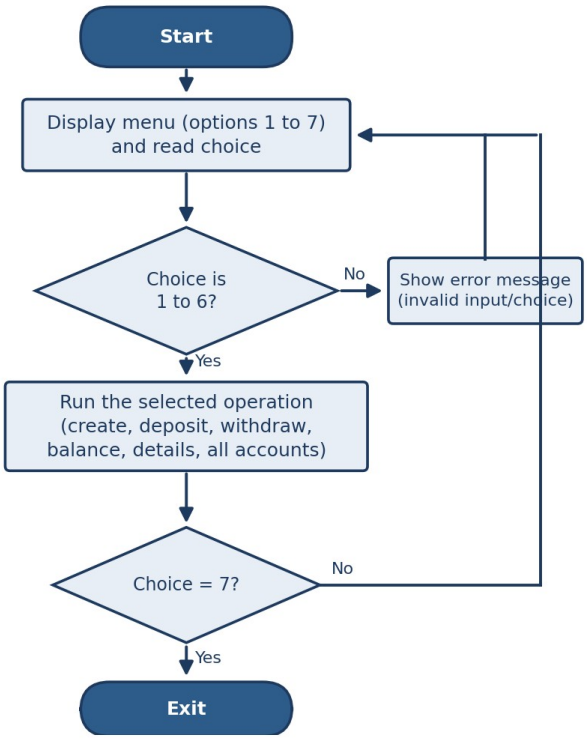


Figure 3: FLOW CHART OF THE MENU

6 Performance Test

A banking application must give correct results every time, because even a small error in a balance is serious. The performance test therefore focused on accuracy, input validation and response time.

Constraints identified:

- Accuracy: balances must be correct after every transaction.
- Rules: a savings account must always keep its minimum balance of 500.
- Robustness: wrong or unexpected input must not crash the program.
- Speed and memory: the program should respond quickly and use little memory.
- Storage: accounts are kept only in memory (an ArrayList), so data is lost when the program exits.

How the constraints were handled in the design: The deposit() and withdraw() methods reject zero or negative amounts and withdrawals that would break the minimum balance. Input is read as text and

converted inside try..catch blocks, so wrong input shows a message instead of stopping the program. The account number is checked before every operation.

Test results around these constraints: All 13 test cases passed, and the whole test session ran in about 0.1 seconds including Java start-up time (details in 6.3).

Constraints not fully tested: Memory use and behaviour with a very large number of accounts were not measured. Because all accounts are held in memory, a very large number of accounts would use more memory, and all data is lost when the program closes.

Recommendation: Store the account data in a file or a database, as listed in the Future work scope.

6.1 Test Plan/ Test Cases

The following test cases were prepared to check accuracy, the banking rules and input handling.

No.	Test case	Input	Expected result
1	Create a savings account	Name Ravi, type Savings, opening balance 5000	Account created, account number 1001
2	Create a current account	Name Priya, type Current, opening balance 1000	Account created, account number 1002
3	Deposit money	Account 1001, amount 2000	New balance 7000.0
4	Withdraw money	Account 1001, amount 1500	New balance 5500.0
5	Withdraw below minimum balance	Account 1001, amount 6000	Error: minimum balance of 500.0 must be maintained
6	Check balance	Account 1001	Current balance 5500.0
7	Unknown account number	Deposit to account 9999	Error: Account not found
8	Negative deposit amount	Account 1001, amount -100	Error: amount must be greater than 0
9	Savings account with low opening balance	Opening balance 100	Error: opening balance of at least 500 needed
10	Text instead of a number in the menu	abc	Message: Please enter a number
11	Menu number out of range	9	Message: Please enter a number from 1 to 7
12	Display one account	Account 1002	Details of Priya, Current, 1000.0
13	Display all accounts	Menu option 6	Both accounts listed with correct balances

6.2 Test Procedure

The program was tested as follows:

- Compile the program with the command: javac BankingInformationSystem.java
- Run it with the command: java BankingInformationSystem
- Give the inputs of the 13 test cases in order, using a prepared input file so that every run uses the same inputs.
- Compare the actual output of each test with the expected result in 6.1.

- Note the time taken for the complete session.

6.3 Performance Outcome

All 13 test cases gave the expected output. The summary is below.

Measure	Result
Test cases run	13
Passed	13
Failed	0
Time for the complete test session	About 0.1 seconds, including Java start-up (Java 21)
Crashes on wrong input	None; every error was shown as a message

7 My learnings

I learned how to turn a problem into a working Java program. The main points of my learning are:

- How to use classes, objects, an interface, an abstract class, inheritance and method overloading in a real program.
- How to use try..catch and input validation so a program handles mistakes without crashing.
- How to prepare test cases, run them and compare the results with what was expected.
- How to write clear weekly reports and learn from quiz mistakes, such as the equals() question in the Week 4 quiz.
- This knowledge will help my career growth because Java and object-oriented design are widely used in software jobs.

8 Future work scope

Ideas that could not be done because of the time limit but can be taken up in future:

- Save the accounts in a file or a database so data is not lost when the program exits.
- Add money transfer between accounts and a transaction history.
- Add a login with an account PIN for security.
- Add interest calculation for savings accounts.
- Build a graphical user interface instead of the console menu.
- Write automatic unit tests for the account classes.